

---

# API Documentation

Chat completions, streaming, and the model catalogue

|          |                                                    |
|----------|----------------------------------------------------|
| Protocol | OpenAI-compatible HTTP + SSE                       |
| Base URL | <code>https://api.example.com/v1</code>            |
| Auth     | <code>Authorization: Bearer &lt;API key&gt;</code> |

Point any OpenAI-compatible client at the base URL and it works unchanged. This document is the complete contract: every field you can send, every field you get back, and every error you can be given.

## Contents

---

- 1. Base URL and endpoints 3
- 2. Authentication 3
- 3. Chat completions 3
- 4. Identifying your end users 5
- 5. The response 5
- 6. Streaming 7
- 7. The model catalogue 8
- 8. Errors 10
- 9. Limits 11
- 10. Examples 11

## 1. Base URL and endpoints

```
base_url = https://api.example.com/v1
```

**Every path works with and without the `/v1` prefix.** Some clients append `/v1` themselves, others are given a base URL that already ends in it. Both hit the same handler, so neither setup is wrong.

| Method  | Path                                                                | Key? | What it does                                          |
|---------|---------------------------------------------------------------------|------|-------------------------------------------------------|
| GET     | <code>/health</code>                                                | no   | Liveness, and how many models are available.          |
| GET     | <code>/v1/models</code><br><code>/models</code>                     | yes  | The model catalogue. See section 7.                   |
| GET     | <code>/v1/models/{id}</code><br><code>/models/{id}</code>           | yes  | One model's entry, or <code>404</code> .              |
| POST    | <code>/v1/chat/completions</code><br><code>/chat/completions</code> | yes  | Chat, streaming and non-streaming. The main endpoint. |
| POST    | <code>/v1/completions</code><br><code>/completions</code>           | yes  | Legacy text completions.                              |
| POST    | <code>/v1/embeddings</code><br><code>/embeddings</code>             | yes  | Embeddings, on models that offer them.                |
| OPTIONS | any of the above                                                    | no   | CORS preflight.                                       |

```
GET /health → {"status":"ok","models":3}
```

## 2. Authentication

Every endpoint except `/health` and CORS preflight takes your API key as a bearer token:

```
Authorization: Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Keys are the literal prefix `Kunci-Zeiko-` followed by 32 characters mixing digits, lower case, upper case and symbols. A key sent bare, with no `Bearer` scheme in front of it, is also accepted — some clients send it that way.

Your key is issued to you directly. Keep it server-side: anyone holding it can spend against your account. If you believe it has been exposed, ask us for a replacement and the old one stops working immediately.

**The model catalogue needs the key too.** `GET /v1/models` is authenticated. A client that fetches the model list before it has been given credentials gets `401`, not an empty list — the usual cause of a mysterious 401 on an otherwise working setup.

## 3. Chat completions

`POST /v1/chat/completions` takes the ordinary OpenAI request body. Two fields are required: `model` and `messages`.

```
{
  "model": "Wissangeni-512B-V1",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"}
  ],
  "stream": true,
  "temperature": 0.7,
  "max_tokens": 512,
  "user_id": "tenant-42:user-9f3c1ab7"
}
```

| Parameter                                                                                                                                                                                                                                                                                                   | Notes                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>model</code>                                                                                                                                                                                                                                                                                          | <b>Required.</b> One of the ids from <code>/v1/models</code> . Matched exactly and case-sensitively.                                                                                                                                                                                                                                                                                                                                          |
| <code>messages</code>                                                                                                                                                                                                                                                                                       | <b>Required.</b> The usual <code>role</code> / <code>content</code> array.                                                                                                                                                                                                                                                                                                                                                                    |
| <code>stream</code>                                                                                                                                                                                                                                                                                         | <code>true</code> returns SSE, <code>false</code> a single JSON body. Every model supports both.                                                                                                                                                                                                                                                                                                                                              |
| <code>user_id</code>                                                                                                                                                                                                                                                                                        | Who this request is for. <b>See section 4 — read it.</b>                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>max_tokens</code><br><code>max_completion_tokens</code>                                                                                                                                                                                                                                               | Clamped down to the model's <code>max_completion_tokens</code> , never refused for being too large. Whichever name you send is the one kept. Omit them and the model's own ceiling applies.                                                                                                                                                                                                                                                   |
| <code>stop</code>                                                                                                                                                                                                                                                                                           | Up to four entries, as OpenAI.                                                                                                                                                                                                                                                                                                                                                                                                                |
| <code>reasoning_effort</code>                                                                                                                                                                                                                                                                               | <code>none</code> , <code>minimal</code> , <code>low</code> , <code>medium</code> , <code>high</code> , <code>max</code> — on models whose catalogue entry has a <code>reasoning</code> block. It can change the price; see <code>pricing</code> in section 7. Also read from <code>reasoning.effort</code> , <code>thinking.effort</code> , the boolean <code>enable_thinking</code> , and a budget in <code>thinking.budget_tokens</code> . |
| <code>reasoning.exclude</code><br><code>include_reasoning</code>                                                                                                                                                                                                                                            | Leave the reasoning trace out of the reply. Either can only remove a trace; neither turns on one a model does not expose.                                                                                                                                                                                                                                                                                                                     |
| <code>temperature</code> , <code>top_p</code> , <code>top_k</code> ,<br><code>min_p</code> , <code>presence_penalty</code> ,<br><code>frequency_penalty</code> ,<br><code>repetition_penalty</code> , <code>seed</code> ,<br><code>logit_bias</code> , <code>logprobs</code> ,<br><code>top_logprobs</code> | Standard sampling and scoring parameters.                                                                                                                                                                                                                                                                                                                                                                                                     |
| <code>response_format</code> , <code>tools</code> ,<br><code>tool_choice</code>                                                                                                                                                                                                                             | JSON mode and tool calling, on models that list them.                                                                                                                                                                                                                                                                                                                                                                                         |
| <code>stream_options</code>                                                                                                                                                                                                                                                                                 | Ignored. Usage always arrives on the final chunk, so <code>include_usage</code> is never needed.                                                                                                                                                                                                                                                                                                                                              |

**Check `supported_parameters` before relying on a parameter.** Each model's catalogue entry lists exactly what it accepts. Sending something not on that list is not an error — it simply has no effect.

## Other endpoints

`/v1/completions` takes the legacy prompt-style body and is billed identically. `/v1/embeddings` takes `model` plus `input` (a string or an array of strings) and returns the standard embeddings response. Neither streams.

## 4. Identifying your end users

**Send one field, `user_id`, on every request.** It is a stable, opaque identifier for the person the request is for.

```
{
  "model": "Wissangeni-512B-V1",
  "messages": [ ... ],
  "user_id": "tenant-42:user-9f3c1ab7"
}
```

That is the whole integration. There is nothing else to set, no header to add, and nothing to configure on your side — we take that one value and handle everything downstream of it.

### Why it matters

`user_id` is the prompt-cache partition key. Two things follow from it, and both are worth money:

- **Isolation.** Requests carrying different `user_id` values can never reuse each other's cached prompt prefix. Without the field, everyone on your API key shares a single partition — one customer's prefix becomes reusable by another customer's request.
- **Cost and speed.** A cached prefix is billed at the cache-read rate, a small fraction of the fresh-input rate, and it removes most of the time-to-first-token on a long system prompt. A `user_id` that changes between requests throws every cache hit away; a stable one keeps them.

You can see it working: `usage.prompt_tokens_details.cached_tokens` appears on the response once a prefix is being reused, and is absent while it is not.

### Choosing the value

| Rule                   | Why                                                                                                                    |
|------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Stable</b>          | The same person gets the same string on every request, across sessions and restarts. This is what makes the cache hit. |
| <b>Unique</b>          | Two different people must never share one value, or they share a cache partition.                                      |
| <b>Opaque</b>          | A hash, a UUID, or an internal account id. <b>Never an email address, phone number, username or real name.</b>         |
| <b>Short and ASCII</b> | Up to 120 characters. Keep well under it.                                                                              |

Omitting `user_id` is allowed and the request still succeeds — you simply get no isolation and, in practice, far fewer cache hits.

## 5. The response

```
{
  "id": "32dadfde-f3dd-4ad8-acc5-387b72415f46",
  "object": "chat.completion",
  "created": 1789936200,
  "model": "Wissangeni-512B-V1",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello! How can I help you today?",

```

```

      "reasoning_content": "The user greeted me...",
      "tool_calls": []
    },
    "finish_reason": "stop"
  }
],
"usage": {
  "prompt_tokens": 18,
  "completion_tokens": 256,
  "total_tokens": 274,
  "prompt_tokens_details": { "cached_tokens": 12 },
  "completion_tokens_details": { "reasoning_tokens": 64 },
  "cost": 0.102949871
}
}

```

| Field                                            | Type    | Meaning                                                                                                                                                                                          |
|--------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>                                  | string  | A UUIDv4 for this request, also returned as the <code>X-Request-Id</code> response header and repeated on every frame of a stream.<br><b>Log it</b> — quoting it lets us find the exact request. |
| <code>object</code>                              | string  | <code>"chat.completion"</code> , or <code>"chat.completion.chunk"</code> on a stream frame.                                                                                                      |
| <code>created</code>                             | integer | Unix seconds, stamped when the request arrived.                                                                                                                                                  |
| <code>model</code>                               | string  | The model id you asked for.                                                                                                                                                                      |
| <code>choices[].message.content</code>           | string  | The answer.                                                                                                                                                                                      |
| <code>choices[].message.reasoning_content</code> | string  | The reasoning trace, on models that expose one. Absent when empty or excluded.                                                                                                                   |
| <code>choices[].message.tool_calls</code>        | array   | Assembled tool calls, each <code>{id, type, function:{name, arguments}}</code> . Absent when there are none.                                                                                     |
| <code>choices[].finish_reason</code>             | string  | <code>stop</code> , <code>length</code> , <code>tool_calls</code> .                                                                                                                              |

## Usage and cost

`prompt_tokens` counts **your own messages only**, measured as they arrived. Anything added on your behalf is not charged to you, so the figure matches what you sent.

| Field                                                         | Type    | Meaning                                                                                                                                                           |
|---------------------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>usage.prompt_tokens</code>                              | integer | Input tokens.                                                                                                                                                     |
| <code>usage.completion_tokens</code>                          | integer | Output tokens, reasoning included.                                                                                                                                |
| <code>usage.total_tokens</code>                               | integer | The two above, added.                                                                                                                                             |
| <code>usage.prompt_tokens_details.cached_tokens</code>        | integer | Cache-hit portion of the input. Never larger than <code>prompt_tokens</code> . <b>Absent when zero.</b>                                                           |
| <code>usage.completion_tokens_details.reasoning_tokens</code> | integer | Reasoning portion of the output. Absent when zero.                                                                                                                |
| <code>usage.cost</code>                                       | number  | What this request cost, in USD, to nine decimal places. Also repeated as <code>usage.usage</code> for clients that read that name — one number, never two prices. |

Nine decimal places is deliberate: a short request at a tenth of a dollar per million tokens comes to a few millionths of a cent, and rounding to six would report it as free. Parse it as a decimal, not a float, if you are reconciling invoices.

## Response headers

| Header                    | Value                                                                                            |
|---------------------------|--------------------------------------------------------------------------------------------------|
| <code>X-Request-Id</code> | Same UUIDv4 as <code>id</code> . On streaming and non-streaming alike.                           |
| <code>Content-Type</code> | <code>application/json</code> , or <code>text/event-stream; charset=utf-8</code> when streaming. |
| <code>Retry-After</code>  | Seconds to wait. Sent with <code>429</code> and <code>503</code> .                               |

## 6. Streaming

Send `"stream": true` and the response is `text/event-stream`: one `data:` frame per chunk, then a final frame carrying usage, then `[DONE]`.

```
data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1",
      "choices":[{"index":0, "delta":{"role":"assistant"}, "finish_reason":null}]}

data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1",
      "choices":[{"index":0, "delta":{"content":"Hel"}, "finish_reason":null}]}

: keep-alive

data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1",
      "choices":[{"index":0, "delta":{"content":"lo!"}, "finish_reason":"stop"}]}

data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1", "choices":[],
      "usage":{"prompt_tokens":18, "completion_tokens":256, "total_tokens":274,
              "cost":0.010293}}

data: [DONE]
```

(Wrapped for print. On the wire each `data:` payload is a single line, and every frame ends with a blank line.)

## What a client must handle

| Frame                    | Rule                                                                                                                                                                                                                                                                                 |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>First frame</b>       | Carries <code>delta.role: "assistant"</code> and usually no content. Do not assume it has text.                                                                                                                                                                                      |
| <b>Content frames</b>    | <code>delta.content</code> is a string fragment; concatenate in arrival order. A reasoning trace, where a model exposes one, arrives as <code>delta.reasoning_content</code> alongside it.                                                                                           |
| <b>Tool calls</b>        | <code>delta.tool_calls</code> arrives as partial fragments keyed by <code>index</code> : accumulate <code>function.arguments</code> across frames before parsing.                                                                                                                    |
| <b>Comment lines</b>     | A line beginning with <code>:</code> is an SSE comment sent to hold the connection open during a quiet moment. <b>Discard it.</b> A compliant SSE parser already does; a hand-rolled one that only reads lines starting <code>data:</code> is also fine. Never try to JSON-parse it. |
| <b>Final usage frame</b> | Has an <b>empty</b> <code>choices</code> <b>array</b> and carries <code>usage</code> . It always arrives. A client that assumes <code>choices[0]</code> exists on every frame will crash here — guard it.                                                                            |

| Frame                     | Rule                                                                                                                                                                                                                                                                                                    |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Error frame</b>        | If a request fails part-way through, a frame carrying <code>{"error": {"message": "...", "type": "server_error"}}</code> is sent before <code>[DONE]</code> , so a stream that has started never just ends in silence. The HTTP status stays <code>200</code> — it was sent before anything went wrong. |
| <code>data: [DONE]</code> | The terminator. Always last, and not JSON.                                                                                                                                                                                                                                                              |

## Practical notes

- `id`, `created` and `model` are identical on every frame of one response.
- Frames can split across TCP reads. Buffer until a blank line before parsing, and accept both `\n\n` and `\r\n\r\n` as the separator.
- A multi-byte character is never split across two frames' `delta.content`.
- Use `curl -N`, and turn off response buffering in your HTTP client.
- Set your client timeout above the maximum request duration in section 9. With keep-alives, an idle-read timeout will not fire spuriously, but a total-request timeout can.
- Disconnecting mid-stream stops the work. You are charged for what was generated up to that point.

## 7. The model catalogue

`GET /v1/models` and `GET /models` return the identical document. `/v1/models/{id}` returns a single entry.

```
curl https://api.example.com/v1/models \
-H "Authorization: Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX"
```

### Before you call it

1. **It needs your API key.** Unauthenticated it answers `401`.
2. **The ids here are the only values `model` accepts.** Matching is exact and case-sensitive.
3. **Re-fetch periodically.** The catalogue reflects what is currently offered and can change; do not cache it for the lifetime of a process.
4. **Prices are decimal strings in USD per single token** — not JSON numbers, and not per million. `"0.000000003"` loses its last digits through a float.
5. **A missing `pricing` object means no published price**, not a free model.

```
{
  "object": "list",
  "data": [
    {
      "id": "Wissangeni-512B-V1",
      "object": "model",
      "name": "Wissangeni 512B V1",
      "display_name": "Wissangeni 512B V1",
      "owned_by": "ZeikoAI",
      "created": 1789900000,
      "description": "Long-context assistant model.",
      "context_length": 128000,
      "max_completion_tokens": 8192,
      "architecture": {
        "modality": "text->text",
        "input_modalities": ["text"],
        "output_modalities": ["text"]
      }
    },
  ],
}
```

```

    "pricing": {
      "prompt": "0.00000015",
      "completion": "0.0000006",
      "input_cache_read": "0.000000003",
      "internal_reasoning": "0.0000006",
      "request": "0",
      "overrides": [
        { "utc_days": ["monday","friday"], "utc_start": 100, "utc_end": 400,
          "prompt": "0.0000003", "completion": "0.0000012",
          "input_cache_read": "0.000000006" }
      ]
    },
    "supported_parameters": ["frequency_penalty","max_tokens","reasoning",
      "reasoning_effort","response_format","seed","stop",
      "temperature","tool_choice","tools","top_p"],
    "default_parameters": { "temperature": 0.7 },
    "reasoning": {
      "mandatory": false,
      "default_enabled": true,
      "supported_efforts": ["max","high","medium","low","none"],
      "default_effort": "high"
    }
  }
}

```

| Field                                         | Meaning                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>                               | The value to put in <code>model</code> .                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>name</code> / <code>display_name</code> | Human-readable label.                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <code>owned_by</code>                         | Who publishes the model.                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>created</code>                          | Unix seconds.                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <code>context_length</code>                   | Input ceiling in tokens. A prompt over it is refused with <code>413</code> , never truncated.                                                                                                                                                                                                                                                                                                                                            |
| <code>max_completion_tokens</code>            | Output ceiling. A larger <code>max_tokens</code> is clamped to this, not refused.                                                                                                                                                                                                                                                                                                                                                        |
| <code>architecture</code>                     | What the model takes in and gives back. <code>modality</code> is the two arrays joined, e.g. <code>"text-&gt;text"</code> .                                                                                                                                                                                                                                                                                                              |
| <code>pricing.prompt</code>                   | Fresh input, USD per token, as a decimal string.                                                                                                                                                                                                                                                                                                                                                                                         |
| <code>pricing.completion</code>               | Output, USD per token.                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <code>pricing.input_cache_read</code>         | Cache-hit input. Absent means a cache hit is <b>not</b> discounted — not that it is free.                                                                                                                                                                                                                                                                                                                                                |
| <code>pricing.input_cache_write</code>        | Cache write, where one is charged.                                                                                                                                                                                                                                                                                                                                                                                                       |
| <code>pricing.internal_reasoning</code>       | Reasoning tokens. Absent means they bill at the output rate.                                                                                                                                                                                                                                                                                                                                                                             |
| <code>pricing.request</code>                  | A flat per-request amount, where there is one.                                                                                                                                                                                                                                                                                                                                                                                           |
| <code>pricing.overrides[]</code>              | <b>Time-of-day pricing.</b> Optional <code>utc_days</code> (lowercase weekday names) and optional <code>utc_start</code> / <code>utc_end</code> as <code>HHMM</code> integers in UTC — <code>100</code> is 01:00, <code>1430</code> is 14:30. Half-open: start inclusive, end exclusive; an end before its start wraps midnight; no times means all day. <b>Each override publishes a complete price</b> , not only the fields it moves. |
| <code>supported_parameters</code>             | Exactly what this model accepts, sorted.                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>default_parameters</code>               | Applied when you do not send them. Override any of them freely.                                                                                                                                                                                                                                                                                                                                                                          |

| Field                  | Meaning                                                                                                                                                                                                                                        |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>reasoning</code> | Present <b>only on models that reason</b> . <code>supported_efforts</code> is the menu <code>reasoning_effort</code> accepts; <code>default_effort</code> is what you get by naming none. Its absence is how you detect a non-reasoning model. |

## Checking a bill against the catalogue

`usage.cost` is the published rates multiplied by the token counts on the same response. Cached input is carved out of `prompt_tokens` and reasoning out of `completion_tokens`, so nothing is counted twice:

```
cost = (prompt_tokens - cached_tokens) * pricing.prompt
      + cached_tokens * pricing.input_cache_read
      + (completion_tokens - reasoning_tokens) * pricing.completion
      + reasoning_tokens * pricing.internal_reasoning
      + pricing.request
```

Use whichever `overrides` entry covers the UTC moment the request ran, and the standing rates otherwise.

## 8. Errors

```
{
  "error": {
    "message": "model \"gpt-9\" is not available",
    "type": "invalid_request_error",
    "code": "model_not_found"
  }
}
```

| Status | code                             | When                                                                                                                     |
|--------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 400    | —                                | Malformed JSON, a missing or empty <code>model</code> , or a body this model will not accept.                            |
| 401    | <code>invalid_api_key</code>     | Missing or unknown key. Also what an unauthenticated <code>GET /v1/models</code> gets.                                   |
| 403    | <code>invalid_api_key</code>     | The key is disabled.                                                                                                     |
| 403    | <code>model_forbidden</code>     | Your key is not entitled to that model.                                                                                  |
| 404    | <code>model_not_found</code>     | Unknown model id.                                                                                                        |
| 404    | <code>not_found</code>           | No such path.                                                                                                            |
| 413    | <code>too_large</code>           | The body is over the size limit, or the prompt is over the model's <code>context_length</code> . The message says which. |
| 429    | <code>rate_limit_exceeded</code> | A per-minute rate limit or a daily quota is spent. The per-minute refusal carries <code>Retry-After</code> .             |
| 502    | <code>model_unavailable</code>   | The model could not answer. Retry with backoff; if it persists, contact us with an <code>X-Request-Id</code> .           |
| 503    | <code>overloaded</code>          | Temporarily at capacity. Carries <code>Retry-After</code> .                                                              |
| 504    | <code>timeout</code>             | The model took too long to answer.                                                                                       |

## Retry policy

- 429, 503 — honour `Retry-After` when present, otherwise exponential backoff from about one second.
- 502, 504 — retry two or three times with backoff.
- 400, 401, 403, 404, 413 — do not retry. Fix the request or the credentials.

## 9. Limits

| Limit                    | Value                                            | Behaviour at the edge                                                                                                                             |
|--------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Request body size        | 20 MB                                            | 413 before the body is read. Comfortably above a full-context request: at typical text density, even a million tokens of JSON is a few megabytes. |
| Input tokens             | per model, as <code>context_length</code>        | 413. Refused, never truncated.                                                                                                                    |
| Output tokens            | per model, as <code>max_completion_tokens</code> | A larger <code>max_tokens</code> is clamped down, not refused.                                                                                    |
| Requests per minute      | per key                                          | 429 with <code>Retry-After</code> .                                                                                                               |
| Requests per day         | per key                                          | 429. Resets daily.                                                                                                                                |
| Tokens per day           | per key                                          | 429. Input and output counted together.                                                                                                           |
| Maximum request duration | 10 minutes                                       | 504. Set your client timeout above this.                                                                                                          |

There is no message-count limit of its own: a conversation is bounded by the model's `context_length` and by the body size above. Your per-key rate limits and quotas are part of your agreement with us — ask if you need them raised.

## 10. Examples

### Streaming, curl

```
curl -N https://api.example.com/v1/chat/completions \
-H "Authorization: Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX" \
-H "Content-Type: application/json" \
-d '{
  "model": "Wissangeni-512B-V1",
  "messages": [{"role": "user", "content": "Hello!"}],
  "user_id": "tenant-42:user-9f3clab7",
  "stream": true
}'
```

### Non-streaming, OpenAI Python SDK

```
from openai import OpenAI

client = OpenAI(
    base_url="https://api.example.com/v1",
    api_key="Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX",
)

reply = client.chat.completions.create(
```

```

    model="Wissangeni-512B-V1",
    messages=[{"role": "user", "content": "Hello!"}],
    extra_body={"user_id": "tenant-42:user-9f3c1ab7"},
)

print(reply.id) # == the X-Request-Id header
print(reply.choices[0].message.content)
print(reply.usage.prompt_tokens, reply.usage.completion_tokens)
print(reply.usage.model_extra.get("cost"))

```

## Streaming, OpenAI Python SDK

```

stream = client.chat.completions.create(
    model="Wissangeni-512B-V1",
    messages=[{"role": "user", "content": "Write a haiku."}],
    extra_body={"user_id": "tenant-42:user-9f3c1ab7"},
    stream=True,
)

usage = None
for chunk in stream:
    if chunk.usage is not None: # the final frame: choices is empty here
        usage = chunk.usage
        continue
    if not chunk.choices: # guard - never assume choices[0] exists
        continue
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="", flush=True)

print("\n", usage)

```

## Streaming, Node with fetch

```

const res = await fetch("https://api.example.com/v1/chat/completions", {
  method: "POST",
  headers: {
    "Authorization": "Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX",
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    model: "Wissangeni-512B-V1",
    messages: [{ role: "user", content: "Hello!" }],
    user_id: "tenant-42:user-9f3c1ab7",
    stream: true,
  }),
});

const reader = res.body.getReader();
const decoder = new TextDecoder();
let buf = "";

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buf += decoder.decode(value, { stream: true });

  // Frames are separated by a blank line; tolerate CRLF from proxies.
  let m;
  while ((m = /\r?\n\r?\n/.exec(buf)) !== null) {
    const frame = buf.slice(0, m.index);
    buf = buf.slice(m.index + m[0].length);

    for (const line of frame.split(/\r?\n/)) {

```

```

    if (line.startsWith(":")) continue;           // keep-alive comment
    if (!line.startsWith("data:")) continue;
    const data = line.slice(5).trim();
    if (data === "[DONE]") continue;             // terminator, not JSON

    const json = JSON.parse(data);
    if (json.error) { console.error(json.error.message); continue; }
    if (json.usage) { console.log("\ncost:", json.usage.cost); }
    const piece = json.choices?.[0]?.delta?.content;
    if (piece) process.stdout.write(piece);
  }
}
}

```

## Listing models

```

curl https://api.example.com/v1/models \
-H "Authorization: Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"

```

**Support.** Every response carries an `X-Request-Id` header, repeated as `id` in the body. Log it. A question that quotes one can be answered precisely; one without usually cannot.